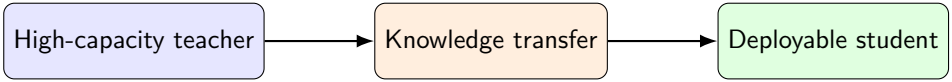
Why Transfer Knowledge?

A strong model may be too expensive to deploy.



Question for this lecture

Can a small network inherit useful decision behavior, features, or relations from a stronger one?

Teacher and Student: Roles

Teacher f_T

- Usually pretrained and accurate
- Often larger, but not required
- Produces logits, features, relations
- Frozen in standard offline KD

Teacher and Student: Roles

Teacher f_T

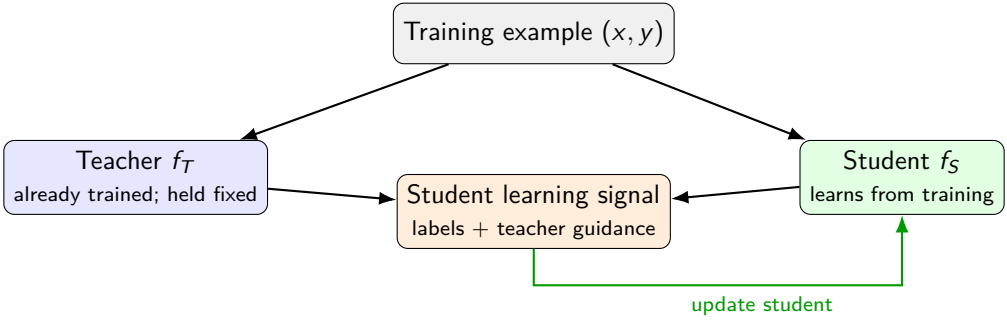
- Usually pretrained and accurate
- Often larger, but not required
- Produces logits, features, relations
- Frozen in standard offline KD

Student f_S

- Target deployment architecture
- Can differ architecturally
- Receives task + teacher signals
- Parameters updated by optimizer

$f_T(x)$ guides $f_S(x)$; **the teacher need not accompany inference.**

Architecture of Knowledge Distillation



After training

Keep and deploy the student; the teacher is no longer needed for inference.

Offline KD: Six Training Steps

1. Train or load a competent **teacher**.
2. Choose the **student** architecture.
3. Load an input minibatch and labels.
4. Compute teacher outputs without teacher gradients.
5. Train the student using both labels and teacher guidance.
6. Backpropagate through student; evaluate and deploy student.

Experiment design

Always compare against a *same-architecture student trained without KD*.

Libraries: The KD Toolchain

Package	Role in Knowledge Distillation
`torch.nn` & `optim`	Model layers, loss modules, and student optimizers
`torch.nn.functional`	Distillation math (`softmax`, `kl_div`, `mse_loss`)
`torch.utils.data`	Minibatch loading via `Dataset` and `DataLoader`
`torchvision` / `transformers`	Pretrained teacher/student architectures and datasets

Environment Setup

```
pip install torch torchvision transformers
```

PyTorch: Define Teacher and Student

```
import torch
from torch import nn
from torchvision import models

teacher = models.resnet18(
    weights=None, num_classes=10)

student = models.mobilenet_v3_small(
    weights=None, num_classes=10)

# weights=None means randomly initialized models.
# First train teacher, THEN freeze it (next slide).
# Student will be optimized during KD training.
```

torchvision.models constructs established architectures; **torch.nn** defines layers and modules.

PyTorch: Train the Teacher First

```
teacher = teacher.to(device)

opt = torch.optim.AdamW(
    teacher.parameters(), lr=1e-3)

for x, y in train_loader:
    x, y = x.to(device), y.to(device)
    teacher.train()
    opt.zero_grad()

    logits = teacher(x)
    loss = torch.nn.functional.cross_entropy(
        logits, y)

    loss.backward()
    opt.step()

# After teacher training, prepare the fixed teacher:
teacher.eval() # inference behavior
teacher.requires_grad_(False) # freeze parameters
```

PyTorch: Train the Teacher First

```
# Later, while training the student:
with torch.no_grad():
    t_logits = teacher(x) # no teacher gradient graph
```

Difference: `eval()` controls dropout/BatchNorm; `requires_grad_(False)` freezes weights; `no_grad()` saves autograd memory.

What Are Hard and Soft Targets?

Ground-Truth Label (Hard)

Husky: [1.00, 0.00, 0.00]

(Husky, Wolf, Car)

Identifies only the single correct class.

Teacher Distribution (Soft)

Husky: [0.65, 0.30, 0.05]

(Husky, Wolf, Car)

Conveys relative similarity across classes.

Dark knowledge: informative probabilities assigned to non-target classes.

Logits and Softmax

Converting Raw Scores to Probabilities

A neural classifier outputs unnormalized **logits** z, converted via Softmax:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- z_i: raw, unbounded score for class i
- $p_i \in [0, 1]$ and $\sum_i p_i = 1$

Numerical Example

Logits $z = [4, 2, 0]$ produce probabilities:

$$p \approx [0.867,\ 0.117,\ 0.016]$$

Temperature: Revealing Small Probabilities

$$p_i^{(\tau)} = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \qquad (\tau > 0)$$

Temperature (τ)	**Husky ($z = 4$)**	**Wolf ($z = 2$)**	**Car ($z = 0$)**
$\tau = 1$ (Standard)	0.867	0.117	0.016
$\tau = 2$ (Softer)	0.665	0.245	0.090
$\tau = 4$ (Softest)	0.506	0.307	0.186

Intuition

Higher τ flattens the distribution, amplifying relative differences among non-target classes.

Distillation Objective

$$\mathcal{L}_{\text{hard}} = \text{CE}(y,\ z_S)$$
$$\mathcal{L}_{\text{soft}} = \tau^2\, D_{\text{KL}}\left(p_T^{(\tau)} \parallel p_S^{(\tau)}\right)$$
$$\mathcal{L}_{\text{total}} = \boxed{(1-\alpha)\,\mathcal{L}_{\text{hard}} + \alpha\,\mathcal{L}_{\text{soft}}}$$

Key Hyperparameters

$\alpha \in [0, 1]$ balances teacher imitation vs. ground truth; τ^2 restores gradient magnitudes scaled down by $1/\tau$.

PyTorch: Distill Knowledge into the Student

```
student = student.to(device)
opt = torch.optim.AdamW(student.parameters(), lr=1e-3)
T, alpha = 4.0, 0.7 # temperature & KD weight

for x, y in train_loader:
    x, y = x.to(device), y.to(device)
    student.train()
    opt.zero_grad()

    with torch.no_grad():
        t_logits = teacher(x)

    s_logits = student(x)
    hard_loss = F.cross_entropy(s_logits, y)
    kd_loss = F.kl_div(
        F.log_softmax(s_logits / T, dim=1),
        F.softmax(t_logits / T, dim=1),
        reduction="batchmean") * (T * T)

    loss = alpha * kd_loss + (1 - alpha) * hard_loss
    loss.backward()
    opt.step()
```

Three Types of Knowledge Transfer

Response-based
Final logits / probabilities

Feature-based
Hidden representations

Relation-based
Distances / similarities

Response-Based KD: Mimicking Output Behavior

- Feed the **same input** x to both teacher and student.
- Match final output logits or softened probabilities.
- **Architecture-agnostic:** works across different model families (e.g., CNN $\rightarrow$ ViT) as long as output classes align.

Example: Husky vs. Wolf vs. Car

Teacher: [0.70, 0.25, 0.05] $\longleftarrow$ Student: [0.42, 0.45, 0.13]

KD pushes the student to prefer Husky over Wolf, while keeping Car low.

Trade-off: Simple and flexible, but ignores intermediate representations.

Response-based KD: What KL Divergence Really Penalizes

$$D_{\mathrm{KL}}(P_T \,\|\, P_S) = \sum_i P_T(i) \log \frac{P_T(i)}{P_S(i)}$$

High $P_T(i)$, Low $P_S(i)$

Heavy penalty: If the teacher thinks Wolf is plausible (0.25) and the student ignores it (0.01), loss spikes.

Low $P_T(i)$, Any $P_S(i)$

Negligible weight: Classes the teacher rules out ($P_T \approx 0$) contribute almost nothing to the sum.

Asymmetry Matters ($D_{\mathrm{KL}}(P_T \| P_S) \neq D_{\mathrm{KL}}(P_S \| P_T)$)

Forward KL puts the teacher P_T first as the reference, forcing the student to cover every class the teacher deems plausible.

Code: Reusable Response-KD Loss Function

```python
import torch.nn.functional as F

def response_kd_loss(s_logits, t_logits, tau=4.0):
    log_ps = F.log_softmax(s_logits / tau, dim=-1)
    pt = F.softmax(t_logits / tau, dim=-1)
    return (tau ** 2) * F.kl_div(
        log_ps, pt, reduction="batchmean")

# Combine with hard-label cross-entropy:
alpha = 0.7
loss = (alpha * response_kd_loss(s_logits, t_logits)
       + (1 - alpha) * F.cross_entropy(s_logits, labels))
```

PyTorch Pitfalls to Avoid: (1) Pass `log_softmax` for student first, `softmax` for teacher second. (2) Use `reduction="batchmean"` so KL averages per sample, not per class.

Code: Implementing Response-KD Loss

```python
import torch.nn.functional as F

def response_kd_loss(s_logits, t_logits, tau=4.0):
    # 1. Soften & convert to probabilities
    log_ps = F.log_softmax(s_logits / tau, dim=-1) # log P_S^(tau)
    pt = F.softmax(t_logits / tau, dim=-1) # P_T^(tau)

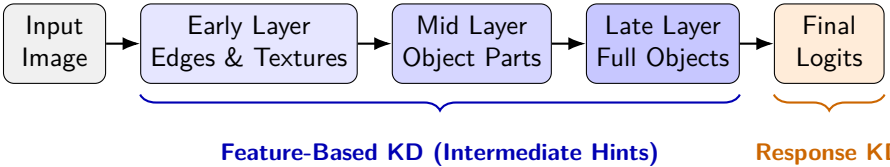
    # 2. Compute forward KL & rescale gradients by tau^2
    kl = F.kl_div(log_ps, pt, reduction="batchmean")
    return (tau ** 2) * kl

# 3. Combine with standard supervised loss
loss = (alpha * response_kd_loss(s_logits, t_logits)
       + (1 - alpha) * F.cross_entropy(s_logits, labels))
```

Why These Exact PyTorch Calls?

- `log_softmax` first, `softmax` second: matches $D_{\mathrm{KL}}(P_T \,\|\, P_S) = \sum P_T(\log P_T - \log P_S)$.
- `reduction="batchmean"`: averages across the batch (default `"mean"` wrongly divides by class count).

Feature-Based KD: Mimicking Intermediate Representations



Core Idea

Instead of only copying the final answer, the student learns *how* the teacher builds representations step-by-step.

Feature KD: Solving the Dimension Mismatch

The Problem

Student feature maps are usually narrower than the teacher's:

$$F_S \in \mathbb{R}^{B \times C_S \times H \times W} \text{ vs. } F_T \in \mathbb{R}^{B \times C_T \times H \times W}$$

(e.g., $C_S = 16$ channels vs. $C_T = 64$ channels)

The Solution: Trainable Adapter A

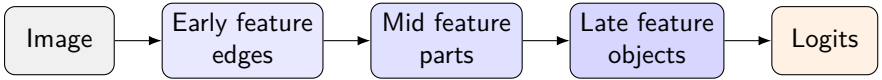
Project student features to match teacher size before computing loss:

$$\hat{F}_S = A(F_S) \in \mathbb{R}^{B \times C_T \times H \times W}$$

$$\mathcal{L}_{\text{feat}} = \frac{1}{BC_THW} \|\hat{F}_S - F_T\|_2^2 \quad \text{(Mean Squared Error)}$$

- **Channel alignment (A):** 1×1 convolution (for images) or a linear projection layer (for Transformers).
- **Spatial alignment:** Use bilinear interpolation or pooling if height/width ($H \times W$) differ.
- **Deployment bonus:** Discard adapter A after training—zero extra inference cost!

Feature Maps: Where Does the Knowledge Live?



Student training signal

Match selected feature maps from the teacher and student, while optimizing the final task.

Feature KD: Alignment and Loss

$$\hat{F}_S = A(F_S), \quad \hat{F}_S \in \mathbb{R}^{B \times C_T \times H \times W}$$

$$\mathcal{L}_{\text{feat}} = \frac{1}{BC_THW} \|\hat{F}_S - F_T\|_2^2$$

Alignment A

- 1×1 convolution
- projection / linear layer
- spatial interpolation

Variants

- feature MSE
- cosine similarity
- attention-map matching

Code: Aligning Features with a 1×1 Conv

```python
import torch.nn as nn
import torch.nn.functional as F

# 1. Define trainable adapter: 16 student channels -> 64 teacher channels
adapter = nn.Conv2d(in_channels=16, out_channels=64, kernel_size=1)

# 2. Project student features and compute MSE against frozen teacher
s_hat = adapter(s_feat) # shape: [B, 64, H, W]
feat_loss = F.mse_loss(s_hat, t_feat) # t_feat computed under no_grad()

# 3. Combine with standard task loss
loss = task_loss + 0.1 * feat_loss
```

Key Takeaways for Training

- **Optimizer:** Include `adapter.parameters()` alongside `student.parameters()` so the adapter learns the projection.
- **Deployment:** Discard `adapter` once training finishes—it is only needed to compute `feat_loss`.

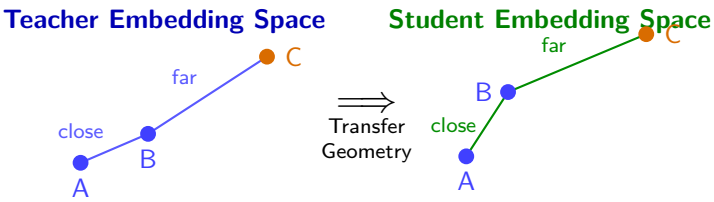
Relation-Based KD: Preserving the Geometry

Feature Matching

Forces $f_S(x_i) \approx f_T(x_i)$ at the exact same coordinates (requires an adapter).

Relation Matching

Preserves *relative distances/angles* between examples (x_i, x_j) across a batch.



Structure: What Relations Can We Transfer?

- **Sample-to-sample (Classification):** Images of the same class should cluster together across a minibatch.
- **Pixel-to-pixel (Segmentation):** Adjacent road or sky pixels should share consistent local similarity.
- **Object-to-object (Detection):** Spatial and semantic relationships between bounding-box regions.
- **Token-to-token (Transformers):** Pairwise attention maps and contextual affinity between words/patches.

Core Intuition

The teacher's internal geometry acts as a relational map of what is similar, structured, or distinct.

Structure: Similarity-Matrix Loss

$$R_T(i,j) = \frac{f_T(x_i)^\top f_T(x_j)}{\|f_T(x_i)\|_2 \, \|f_T(x_j)\|_2} \quad \text{(Cosine Similarity)}$$

$$\mathcal{L}_{\text{rel}} = \frac{1}{B^2} \sum_{i=1}^{B} \sum_{j=1}^{B} \Big(R_T(i,j) - R_S(i,j) \Big)^2$$

Key Advantage

R_T and R_S are both $B \times B$ matrices—**no adapter needed** even if $D_S \neq D_T$.

Computational Cost

Computing all pairs scales as $\mathcal{O}(B^2)$ per batch (costly for large B or dense pixels).

Code: Relation Loss in Python

```python
import torch.nn.functional as F

def relation_loss(s_feat, t_feat):
    # s_feat: [B, D_S], t_feat: [B, D_T] (D_S and D_T can differ!)
    s = F.normalize(s_feat, dim=1) # [B, D_S] unit vectors
    t = F.normalize(t_feat, dim=1) # [B, D_T] unit vectors
    sim_s = s @ s.T # [B, B] student similarity
    sim_t = t @ t.T # [B, B] teacher similarity
    return F.mse_loss(sim_s, sim_t)
```

Note: Assumes `t_feat` is computed inside `torch.no_grad()`. Matrix multiplication (`@`) computes all B^2 pairwise cosine similarities at once.

Summary: Which KD Type Should We Choose?

KD Type	What Matches	Requires Adapter?	Key Trade-off
Response	Output logits	No (if classes match)	Easy baseline; ignores internal reasoning
Feature	Hidden layers	**Yes** (if shapes differ)	Rich intermediate hints; extra tuning
Relation	Pairwise geometry	No ($B \times B$ matrices)	Architecture-free; $\mathcal{O}(B^2)$ batch cost

Empirical Best Practice: Ablate Incrementally

Never stack every loss blindly. Always benchmark step-by-step under the same compute budget:

$$\textbf{CE only} \longrightarrow \textbf{CE + Response} \longrightarrow \textbf{CE + Response + Feature/Relation}$$

Exercise 1: Classification with Mismatched Layers

Scenario

Scenario

You are distilling a **ResNet-50** teacher (2048 hidden channels) into a **MobileNet** student (576 hidden channels) on a 10-species wildlife dataset, using **both logit and intermediate feature distillation**.

1. Which loss uses the ground-truth wildlife labels?
2. Which loss transfers the teacher's dark knowledge across species?
3. How do you align the 576-channel student layer to the 2048-channel teacher?
4. What is the minimum ablation needed to prove feature distillation helped?

Exercise 1: Classification with Mismatched Layers

Scenario

Scenario

You are distilling a **ResNet-50** teacher (2048 hidden channels) into a **MobileNet** student (576 hidden channels) on a 10-species wildlife dataset, using **both logit and intermediate feature distillation**.

1. Which loss uses the ground-truth wildlife labels?
2. Which loss transfers the teacher's dark knowledge across species?
3. How do you align the 576-channel student layer to the 2048-channel teacher?
4. What is the minimum ablation needed to prove feature distillation helped?

Solution Design

Ground truth: Hard-label Cross-Entropy ($\mathcal{L}_{\text{CE}}$). **Dark knowledge:** Response KD via temperature-scaled KL divergence ($\mathcal{L}_{\text{resp}}$). $\mathcal{L} = (1-\alpha)\mathcal{L}_{\text{CE}} + \alpha\mathcal{L}_{\text{resp}} + \lambda\mathcal{L}_{\text{feat}}$, using a 1×1 **Conv adapter** ($576 \rightarrow 2048$).

Exercise 2: Cross-Architecture Retrieval Embeddings

Scenario

You want a tiny **CNN** student (128-dim output embedding) to mimic a giant **Vision Transformer** teacher (768-dim output embedding) for visual product search—*with no class labels or classification head*.

1. Why can't we use standard Response KD (logit KL divergence) here?
2. Why is layer-by-layer Feature KD awkward between a ViT and a CNN?
3. Which KD family works cleanly here without needing a projection adapter?
4. What happens to loss computation cost if you increase batch size from 32 to 128?

Exercise 2: Cross-Architecture Retrieval Embeddings

Scenario

You want a tiny **CNN** student (128-dim output embedding) to mimic a giant **Vision Transformer** teacher (768-dim output embedding) for visual product search—*with no class labels or classification head*.

1. Why can't we use standard Response KD (logit KL divergence) here?
2. Why is layer-by-layer Feature KD awkward between a ViT and a CNN?
3. Which KD family works cleanly here without needing a projection adapter?
4. What happens to loss computation cost if you increase batch size from 32 to 128?

Solution Design

Use **Relation-Based KD** ($\mathcal{L}_{\text{rel}}$) on L2-normalized embeddings!

Both produce $B \times B$ cosine similarity matrices (no adapter needed). Increasing B by $4\times$ increases pairwise computations by $4^2 = \mathbf{16}\times$.

Training Schemes: Who, When, and What Data?

Rotating Our Perspective

Section II — What is transferred? $\longrightarrow$ Response / Feature / Relation

Section III — How is training organized? $\longrightarrow$ Offline / Online / Self / Data-Free

- **Orthogonal axes:** Any training scheme in Section III can use any loss from Section II.
- **What changes:** Who plays the teacher, when the teacher is trained, and what data is available.
- **Baseline reference:** Everything we coded so far used conventional **Offline KD** (frozen pre-trained teacher).

Offline KD: The Two-Stage Reference Setup



- **Upfront teacher cost:** Teacher is trained once (or downloaded from a model zoo).
- **Stable targets:** Teacher predictions never drift while the student trains.
- **Practical trade-off:** Easiest to reproduce, but requires hosting a large teacher first.

Data-Free KD: When the Dataset Is Missing

The Constraint

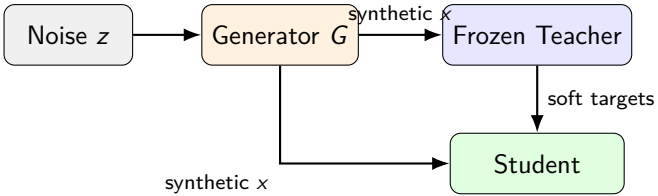
You have a trained **teacher checkpoint**, but the original training dataset is unavailable.

- **Why it happens:** Proprietary medical/financial records, deleted logs, or strict licensing.
- **Core workaround:** Replace missing data with **synthetic inputs** (generated from noise) or unrelated public proxy data.
- **How it works:** Query the frozen teacher on synthetic inputs and train the student to match its outputs.

Key Insight

"Data-free" means no access to the *original* dataset—the student still needs synthetic or proxy inputs to query the teacher!

Data-Free KD: Generator–Teacher–Student Loop



Two Alternating Steps

1. **Train Generator** G**:** Synthesize diverse inputs $x = G(z)$ that strongly activate the teacher.
2. **Train Student** S**:** Match the teacher's predictions on those synthetic inputs x.

Code: Updating the Synthetic Generator

```
# Step 1 only: train Generator G (teacher weights stay frozen)
teacher.requires_grad_(False)
z = torch.randn(16, 64, device=device)
x_synth = generator(z)
target_cls = torch.randint(0, 10, (16,), device=device)

logits = teacher(x_synth) # NO torch.no_grad() here!
loss_g = F.cross_entropy(logits, target_cls)
gen_opt.zero_grad()
loss_g.backward() # gradients flow back to G
gen_opt.step()
```

Subtlety: Teacher weights are frozen (`requires_grad_(False)`), but we omit `torch.no_grad()` so gradients flow *through* the teacher back into `generator`.

Self-Distillation: When Teacher and Student Match

Core Definition

The teacher and student share the **exact same architecture and parameter count**.

- **Different goal:** Improves **accuracy and generalization** rather than compressing model size.
- **Why it works:** Soft teacher targets act as adaptive label smoothing, preventing overfitting.
- **Landmark paper:** *Born-Again Neural Networks* (Furlanello et al., 2018)—a student trained from scratch beats its identical teacher!

Self-Distillation: Three Practical Forms

Form	Who Plays the Teacher?	Classic Example
Generational	Frozen checkpoint from a prior run	Born-Again Networks
Internal	Deeper layers / main classifier head	Early-exit branches
Temporal	Exponential Moving Average (EMA) of past weights	Mean Teacher / DINO

Caution: Confirmation Bias

If a model teaches itself unchecked, it can reinforce its own wrong predictions—always combine with ground-truth labels or strong data augmentation.

Code: Generational Self-Distillation

```
import copy
teacher = copy.deepcopy(base_model).eval()
teacher.requires_grad_(False)
student = models.resnet18(weights=None, num_classes=10).to(device)
for x, y in train_loader:
with torch.no_grad():
t_logits = teacher(x)
s_logits = student(x)
loss = (0.5 * response_kd_loss(s_logits, t_logits)
+ 0.5 * F.cross_entropy(s_logits, y))
```

Continuity: Reuses `response_kd_loss` from Slide 16—only the teacher's origin changed!

Appendix: Complete Offline KD Minibatch

```
teacher.eval(); student.train()
for x, y in loader:
    x, y = x.to(device), y.to(device)
    with torch.no_grad():
        logits_t = teacher(x)
    logits_s = student(x)
    hard = F.cross_entropy(logits_s, y)
    soft = kd_loss(logits_s, logits_t, tau=4.0)
    loss = 0.6 * hard + 0.4 * soft
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

See `kd_cifar10_lab.py` for a complete runnable example.

Appendix: Reproducible Experiments

- Same student initialization and data split across CE and KD baselines.
- Vary τ, loss weights, teacher quality, and adapter design.
- Track classification accuracy or mAP/mIoU, peak memory, latency, and parameters.
- Check teacher mistakes; do not report synthetic teacher superiority.
- Run the companion `kd_cifar10_lab.py` with `-data fake` for a code sanity check, not a scientific result.

Appendix: References (Core Papers)

- Hinton, Vinyals, Dean (2015), *Distilling the Knowledge in a Neural Network*. https://arxiv.org/abs/1503.02531
- Romero et al. (2015), *FitNets*. https://arxiv.org/abs/1412.6550
- Park et al. (2019), *Relational Knowledge Distillation*. https://arxiv.org/abs/1904.05068
- Furlanello et al. (2018), *Born Again Neural Networks*. https://arxiv.org/abs/1805.04770
- Zhang et al. (2018), *Deep Mutual Learning*. https://arxiv.org/abs/1706.00384
- Yin et al. (2020), *Dreaming to Distill / DeepInversion*. https://arxiv.org/abs/1912.08795
- Yang et al. (2022), *Focal and Global KD for Detectors*. https://arxiv.org/abs/2111.11837
- Liu et al. (2019), *Structured KD for Semantic Segmentation*. https://arxiv.org/abs/1903.04197
- Agarwal et al. (2024), *On-Policy Distillation of Language Models*. https://arxiv.org/abs/2306.13649

Appendix: Official Implementation Guides

- PyTorch Knowledge Distillation Tutorial:
  `https://docs.pytorch.org/tutorials/beginner/knowledge_distillation_tutorial.html`
- KL divergence API:
  `https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.kl_div.html`
- TorchVision Feature Extraction:
  `https://docs.pytorch.org/vision/main/feature_extraction.html`
- TorchVision Detection Fine-Tuning:
  `https://docs.pytorch.org/tutorials/intermediate/torchvision_tutorial.html`
- Hugging Face Transformers Causal LM:
  `https://huggingface.co/docs/transformers/tasks/language_modeling`